Vad fan ska vi handla

Backend and hosting with Firebase

How to replace the mock backend with Firebase Authentication, Cloud Firestore and Cloud Functions, share lists between households, send push notifications, build the Android app with EAS and host the web version on GitHub Pages. With code that matches the app's current api/ functions.

October 2026 · replaces the Supabase edition of this guide

Contents

1. Where the app is today 3
2. The big picture 4
3. What the backend has to do 5
4. Why Firebase, and how to use it 7
5. Setting up Firebase 8
6. Connecting the app 11
7. Live updates 18
8. Push notifications 21
9. Testing with the emulators 25
10. Getting the app onto phones 26
11. Web on GitHub Pages 28
12. Hosting and what it costs 30
13. Step-by-step roadmap 31
14. Security checklist 33
15. Glossary 34

1. Where the app is today

Every screen works: you can log in, switch between lists, add items with amount and category, check them off, manage lists, change language and appearance, and get notifications. But everything behind the screens is pretend. The app runs against a mock backend that lives inside the app itself.

Part	Status
Screens, components, theme	Done. Real code you keep.
Contexts and hooks (AuthContext, ListsContext, useItems …)	Done. Real code you keep, with small changes described in chapter 6–7.
api/ folder	The seam. Today each file is one line: `export * from "@/mock/api/…"`. This is what you rewrite.
mock/ folder	Fake database (mock/db.ts), fake network delay, and a simulated second user. Deleted at the end.

Because the mock lives in the app's memory, these things don't work for real yet:

- Sharing. Each phone has its own copy of the data. Nothing you add shows up anywhere else.
- Saving. Everything resets when the app reloads.
- Accounts. Login accepts any password for two hard-coded test emails. Nobody can sign up.
- Households. Every user sees every list. There is no idea of "my lists" or of inviting someone.

> **The good news**
>
> The rest of the app only talks to the functions in api/, and those already have the right names and types: `login`, `getLists`, `addItem`, `toggleItem`, `setAllDone` and so on. Replacing their insides with Firebase calls is most of the work, and almost every signature can stay exactly as it is. The few places where something has to be added or changed are called out in this guide, so you can agree on them before touching the code.

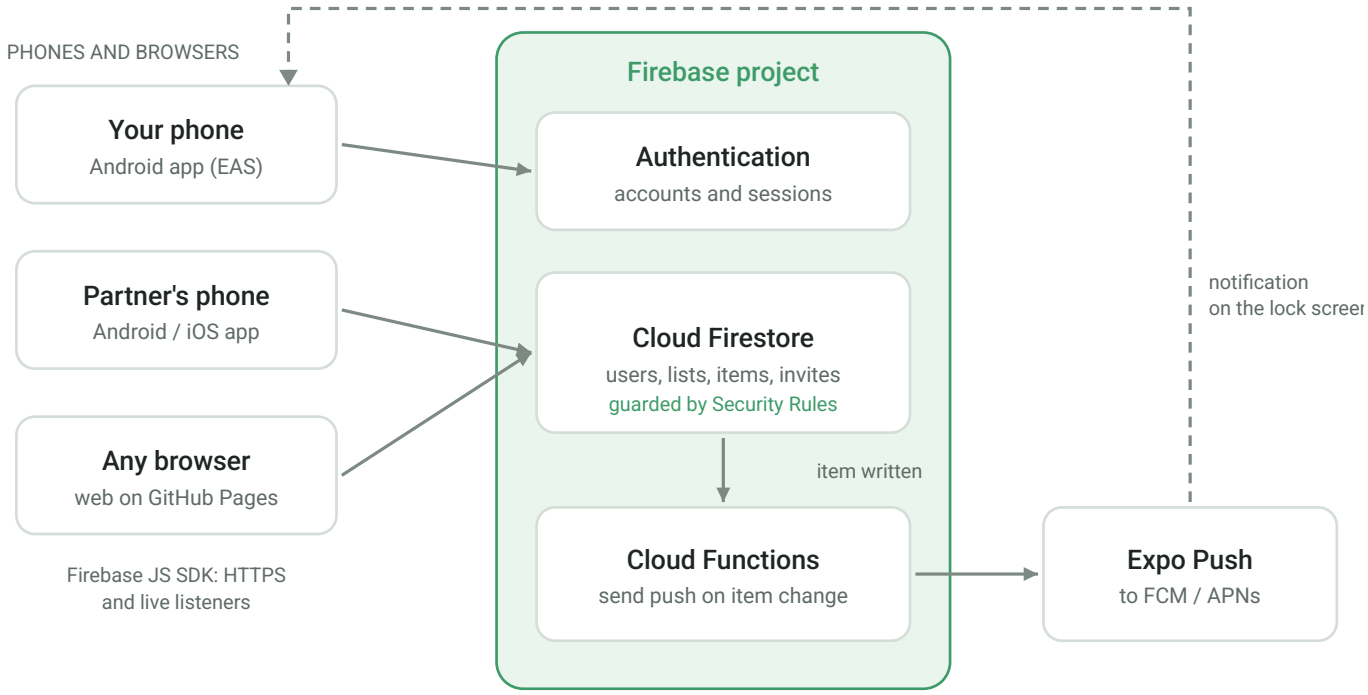
1.1 The current api/ surface

This is what the rest of the app calls today. Chapter 6 rewrites each of these.

File	Functions (as they are now)
auth.ts	`login(email, password): LoginResponse`, `getCurrentUser(token): User`
settings.ts	`updateSettings(userId, settings): User`
users.ts	`getUsers(): User[]` (only used to put names on notifications)
lists.ts	`getLists()`, `createList(name)`, `deleteList(listId)`
items.ts	`getItems(listId)`, `addItem(request, userId)`, `toggleItem(itemId, userId)`, `setAllDone(listId, done, userId)`, `removeDone(listId)`
activity.ts	`getActivity(afterId)`, polled every 10 s by hooks/useActivityNotifications.ts

2. The big picture

A shopping list you share needs a place on the internet where the lists live, so every phone in the household reads and writes the same data. With Firebase, that place is a Firebase project. The app talks to it directly through the Firebase JavaScript SDK; there is no server of your own.



Piece	What it does	Chapter
Firebase Authentication	Accounts: sign up, log in, stay logged in.	5–6
Cloud Firestore	Stores users, lists, items and invites. Security Rules decide who may read and write what.	5
Realtime listeners	Firestore pushes changes to every open app the moment they happen. Replaces polling.	7
Cloud Functions	Server code that runs when an item changes and sends push notifications.	8
EAS	Builds the Android (and later iOS) app.	10
GitHub Pages	Hosts the web version, a static export of the same code.	11

3. What the backend has to do

It helps to be specific before choosing tools. Most of this already exists in the mock, so it's a checklist rather than a wish list. The new part is households: lists belong to the people who were invited to them, not to everyone.

3.1 Accounts

Anyone can create an account with email and password. Logging in gives the app a session that the SDK stores on the phone and refreshes by itself. Sign-in with Google or Apple can come later.

3.2 Lists and households

A "household" is not a separate thing in the database. It is simply the set of people who are members of a list. Whoever creates a list is its owner and first member. They invite others with an invite code (or a link containing it). Accepting the invite adds you to the list's members. This is more flexible than a fixed household: you can share "Veckohandling" with your partner and "Bygghandeln" with a friend.

3.3 Data

Matching types/models.ts, plus what's needed for sharing and push:

Collection	Contents	Who can see it
users/{uid}	name, email, settings (language, theme, notifications)	Only that user
profiles/{uid}	name only	Any logged-in user who knows the id (used for "Anna added Milk")
lists/{listId}	name, ownerId, members (array of uids), createdAt	Members of the list
items/{itemId}	listId, name, amount, category, done, addedBy, updatedBy, createdAt	Members of the item's list
invites/{code}	listId, listName, createdBy, createdAt	Anyone who has the code
users/{uid}/pushTokens/{token}	platform, updatedAt	Only that user (and the Cloud Function)

Two fields are new compared with the mock. `createdAt` lets the database return items in the order they were added, which `sortItems` in helpers/list.ts relies on. `updatedBy` records who made the latest change, so a notification can say who checked something off and never notify the person who did it.

3.4 Realtime

When someone adds "Milk" at home, the list in the shop should update by itself. Today the app polls every 10 seconds. Firestore listeners (`onSnapshot`) deliver changes over an open connection instead: faster, and cheaper in both battery and reads.

3.5 Server-side logic

One thing must happen on a server: when someone adds or checks off an item, the other members get a push notification, even if their app is closed. That is a Cloud Function triggered by the change in Firestore (chapter 8).

4. Why Firebase, and how to use it

4.1 What Firebase gives this app

- Auth, database, realtime and server functions in one project, managed from one console and one CLI.
- Realtime is built into the database. Every query can be a live listener, with no extra setup.
- Offline handling out of the box. Writes made in the shop basement are queued and sent when the signal returns, and the screen updates immediately.
- The same Firebase project provides FCM, which Android push needs anyway (chapter 10.4).
- Generous free tier (Spark plan) for everything except Cloud Functions.

4.2 What to keep in mind

- Firestore is a document database, not SQL. There are no joins and no cascading deletes. The data model in chapter 5 is designed around the queries the app makes, and the api/ code deletes a list's items itself.
- Security Rules replace RLS. The Firebase config in the app is public by design. What protects the data is the rules file. It lives in the repo as firestore.rules and is deployed with the CLI.
- Cloud Functions require the Blaze (pay-as-you-go) plan, which needs a card on file. Its free quota is far more than this app will use, but set a budget alert anyway. Everything up to push notifications works on the free Spark plan.

4.3 Use the JavaScript SDK, not React Native Firebase

There are two ways to use Firebase from Expo:

Option	Expo Go	Web	Verdict
firebase (the official JavaScript SDK)	Works	Works, same code	Use this.
@react-native-firebase	Doesn't work: needs a development build	Not supported	Not for this project

The project rules say Expo Go must keep working and that web is a first-class target. The JavaScript SDK does both, with one code path for every platform.

> **Update CLAUDE.md first**
>
> CLAUDE.md currently names Supabase as the backend ("Don't switch away from Supabase…", RLS, Supabase keys in .env.local) and points to this guide. Before anyone, person or agent, starts on the code, update CLAUDE.md to say Firebase: Firestore Security Rules instead of RLS, Firebase config in .env.local, no Firebase calls outside api/. Otherwise the instructions contradict this guide.

5. Setting up Firebase

5.1 Create the project

1. Create a project at console.firebase.google.com. Google Analytics isn't needed.
2. Build › Authentication › Get started. Enable the Email/Password provider. Leave email enumeration protection on (the default for new projects).
3. Build › Firestore Database › Create database. Choose production mode (everything denied until you deploy rules) and a location close to you, for example europe-north2 (Stockholm), which this project uses. The location can't be changed later. Cloud Functions isn't offered in every Firestore region (not in europe-north2, for one); the function in chapter 8 then runs in the nearest region that has it, europe-north1 (Finland), while its trigger stays with the database.
4. Project settings › General › Your apps › Add app › Web (</>). Name it "vadfanskavihandla". Copy the `firebaseConfig` values; the app needs them. This one web config is used on Android, iOS and web alike, because all three run the JavaScript SDK.

> **Never use "test mode"**
>
> Test mode creates rules that let anyone read and write everything for 30 days. Start in production mode and deploy the real rules below before the app connects.

5.2 Install the CLI and put the backend config in the repo

The Firebase CLI deploys rules, indexes and functions from files in the repo, so they are reviewed and versioned like the rest of the code.

```
npm install -g firebase-tools
firebase login
firebase init firestore functions emulators
#   Firestore: use firestore.rules and firestore.indexes.json
#   Functions: TypeScript, in functions/
#   Emulators: Authentication, Firestore, Functions
```

This creates firebase.json, .firebaserc, firestore.rules, firestore.indexes.json and a functions/ folder with its own package.json. Add `functions/node_modules/` and `functions/lib/` to .gitignore.

The functions/ folder is a separate Node project that never ships in the app. Exclude it from the app's TypeScript and lint runs: add `"functions"` to `exclude` in tsconfig.json and to the ignore list in eslint.config.js, so `npx tsc --noEmit` and `npm run lint` keep checking only the app.

5.3 Security Rules

This is the most important file in the backend. It is the only thing between your data and anyone who opens the web app's source and copies the config. Everything not explicitly allowed is denied.

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    function signedIn() { return request.auth != null; }
    function uid() { return request.auth.uid; }
    function list(listId) {
```

```
      return get(/databases/$(database)/documents/lists/$(listId));
    }
    function isMember(listId) {
      return signedIn() && uid() in list(listId).data.members;
    }
    function changes() {
      return request.resource.data.diff(resource.data).affectedKeys();
    }

    // Private: email and settings.
    match /users/{userId} {
      allow read, write: if signedIn() && uid() == userId;
      match /pushTokens/{token} {
        allow read, write: if signedIn() && uid() == userId;
      }
    }

    // Public name, so others can see who added what.
    // "get" only: ids can be looked up but the collection can't be listed.
    match /profiles/{userId} {
      allow get: if signedIn();
      allow create, update: if signedIn() && uid() == userId
        && request.resource.data.keys().hasOnly(['name']);
    }

    match /lists/{listId} {
      allow read: if signedIn() && uid() in resource.data.members;
      allow create: if signedIn()
        && request.resource.data.ownerId == uid()
        && request.resource.data.members == [uid()];
      allow update: if signedIn() && (
        // Members may rename the list.
        (uid() in resource.data.members && changes().hasOnly(['name']))
        // Anyone with a valid invite may add themselves, and only themselves.
        || (changes().hasOnly(['members', 'invite'])
            && request.resource.data.members == resource.data.members.concat([uid()])
            && get(/databases/$(database)/documents/invites/$(request.resource.data.invite))
                 .data.listId == listId)
        // Members may leave.
        || (uid() in resource.data.members && changes().hasOnly(['members'])
            && request.resource.data.members == resource.data.members.removeAll([uid()]))
      );
      allow delete: if signedIn() && resource.data.ownerId == uid();
    }

    match /items/{itemId} {
      allow read, delete: if isMember(resource.data.listId);
      allow create: if isMember(request.resource.data.listId)
        && request.resource.data.addedBy == uid()
        && request.resource.data.updatedBy == uid();
      allow update: if isMember(resource.data.listId)
        && request.resource.data.listId == resource.data.listId
        && request.resource.data.updatedBy == uid();
    }

    // Invite codes are long random document ids. Knowing one is the permission.
    match /invites/{code} {
      allow get: if signedIn();
      allow create: if isMember(request.resource.data.listId)
        && request.resource.data.createdBy == uid();
      allow delete: if signedIn() && resource.data.createdBy == uid();
    }
  }
}
```

A few things worth understanding:

- Rules are not filters. A query is only allowed if the rules can prove every possible result is allowed. That is why

`getItems` must query `where("listId", "==", listId)`: the rule can then check membership of that one list. A query for "all items" is rejected outright, not filtered.

- `updatedBy == uid()` means nobody can pretend someone else made a change, so notifications can trust it.
- Only the owner can delete a list. Other members can leave it. The mock lets anyone delete any list; the delete button can stay, and a non-owner gets an error (or hide it for non-owners later).

Deploy with `firebase deploy --only firestore:rules`.

5.4 Indexes

Firestore needs a composite index for queries that filter on one field and sort on another. Put these in firestore.indexes.json and deploy with `firebase deploy --only firestore:indexes`:

```
{
  "indexes": [
    {
      "collectionGroup": "lists",
      "queryScope": "COLLECTION",
      "fields": [
        { "fieldPath": "members", "arrayConfig": "CONTAINS" },
        { "fieldPath": "createdAt", "order": "ASCENDING" }
      ]
    },
    {
      "collectionGroup": "items",
      "queryScope": "COLLECTION",
      "fields": [
        { "fieldPath": "listId", "order": "ASCENDING" },
        { "fieldPath": "createdAt", "order": "ASCENDING" }
      ]
    }
  ]
}
```

If you forget one, the query fails with an error containing a link that creates the missing index. Add it to the file afterwards so the repo stays the source of truth.

5.5 Test the rules locally first

`firebase emulators:start` runs Auth, Firestore and Functions on your machine, with a UI at localhost:4000. Point the app at it during development (chapter 6.2) and try the cases that matter: two users sharing a list both see it; a third user doesn't; accepting an invite works; a forged `addedBy` is rejected. Nothing you do there touches the real project or costs anything.

6. Connecting the app

6.1 Install the SDK

Two new dependencies. CLAUDE.md requires asking before adding them, so get the go-ahead first. Use `npx expo install` so versions match the Expo SDK:

```
npx expo install firebase @react-native-async-storage/async-storage
```

AsyncStorage is where the Firebase SDK keeps the login on phones. (On web it uses the browser's own storage.) Both packages are pure JavaScript, so Expo Go keeps working.

Put the web config from chapter 5.1 in .env.local in the project root. Expo exposes variables starting with `EXPO_PUBLIC_` to the app, and .gitignore already excludes `.env*.local`:

```
EXPO_PUBLIC_FIREBASE_API_KEY=AIza...
EXPO_PUBLIC_FIREBASE_AUTH_DOMAIN=<project-id>.firebaseapp.com
EXPO_PUBLIC_FIREBASE_PROJECT_ID=<project-id>
EXPO_PUBLIC_FIREBASE_APP_ID=1:...:web:...
# Optional: talk to the local emulators instead of the real project.
EXPO_PUBLIC_FIREBASE_EMULATOR_HOST=
```

> **The Firebase config is not a secret**
>
> These values identify your project; they don't grant access. They end up in the web bundle and the APK no matter what. Access is controlled by Security Rules. The only Firebase credential that is a real secret is a service account key, and that never goes in the app or the repo.

6.2 One shared client: api/firebase.ts

Every other api/ file imports from here, and nothing outside api/ imports from firebase/*.

```
// The Firebase app, auth and Firestore instances the rest of api/ uses.
// Nothing outside api/ imports from firebase/*.
import AsyncStorage from "@react-native-async-storage/async-storage";
import { getApp, getApps, initializeApp } from "firebase/app";
import { ApiError } from "@/api/ApiError";
import * as firebaseAuth from "firebase/auth";
import { Auth, connectAuthEmulator, getAuth, initializeAuth, Persistence } from "firebase/auth";
import { connectFirestoreEmulator, initializeFirestore } from "firebase/firestore";
import { Platform } from "react-native";

// Exported by the React Native build of firebase/auth but missing from its types. Read here rather
// than with @ts-expect-error on an import, which "organize imports" on save can separate from it.
const { getReactNativePersistence } = firebaseAuth as unknown as {
  getReactNativePersistence: (storage: typeof AsyncStorage) => Persistence;
};
// Fast Refresh re-runs this file; initializing twice throws.
const isFirstLoad = getApps().length === 0;
const app = isFirstLoad
  ? initializeApp({
    apiKey: process.env.EXPO_PUBLIC_FIREBASE_API_KEY,
    authDomain: process.env.EXPO_PUBLIC_FIREBASE_AUTH_DOMAIN,
    projectId: process.env.EXPO_PUBLIC_FIREBASE_PROJECT_ID,
    appId: process.env.EXPO_PUBLIC_FIREBASE_APP_ID,
```

```
  })
  : getApp();
// The browser keeps the session by itself; on phones it's stored in AsyncStorage.
export const auth: Auth =
  Platform.OS === "web" || !isFirstLoad
    ? getAuth(app)
    : initializeAuth(app, { persistence: getReactNativePersistence(AsyncStorage) });
// Optional fields like amount are undefined, which Firestore rejects unless told to skip them.
export const db = initializeFirestore(app, { ignoreUndefinedProperties: true });
const emulatorHost = process.env.EXPO_PUBLIC_FIREBASE_EMULATOR_HOST;
if (emulatorHost && isFirstLoad) {
  connectAuthEmulator(auth, `http://${emulatorHost}:9099`);
  connectFirestoreEmulator(db, emulatorHost, 8080);
}
// The logged-in user's id, for queries that only make sense for "me".
export function currentUid(): string {
  const uid = auth.currentUser?.uid;
  if (!uid) throw new ApiError("errors.invalidSession");
  return uid;
}
```

Two notes on that file:

- The cast for `getReactNativePersistence` works around a long-standing gap in Firebase's TypeScript types: the function exists at runtime in the React Native build but isn't in the type definitions (firebase-js-sdk issue #9316). A `@ts-expect-error` comment above the import also works, until "organize imports" on save moves another import between the comment and its line, which breaks `tsc`. The cast can't be separated that way.
- Emulator host: on the Android emulator, your computer is `10.0.2.2`, not `localhost`.

6.3 Errors carry translation keys

The app's rule is that errors carry an `ErrorKey`, not a sentence. Firebase throws its own errors with a `code`. Translate them in one place, also in api/firebase.ts, and wrap calls that the user can trigger:

```
// Maps Firebase error codes to the app's translated error keys.
export function toApiError(error: unknown): ApiError {
  if (error instanceof ApiError) return error;
  switch ((error as { code?: string }).code) {
    case "auth/invalid-credential":
    case "auth/invalid-email":
      return new ApiError("errors.invalidCredentials");
    case "auth/email-already-in-use":
      return new ApiError("errors.emailInUse");
    case "auth/weak-password":
      return new ApiError("errors.weakPassword");
    case "permission-denied":
      return new ApiError("errors.notAllowed");
    case "unavailable":
    case "auth/network-request-failed":
      return new ApiError("errors.offline");
    default:
      return new ApiError("errors.unknown");
  }
}
```

Add the new keys (`invalidCredentials`, `emailInUse`, `weakPassword`, `notAllowed`, `offline`, `unknown`, `inviteNotFound`) under `errors` in both en.json and sv.json. `unknownEmail` and the test-account hint can go once the mock is gone. With email enumeration protection on, Firebase deliberately doesn't say whether the email or the password was wrong, hence one `invalidCredentials` key.

6.4 api/auth.ts

`login` and `getCurrentUser` keep their signatures. Two functions are added; adding doesn't break any caller, but `logout` must be wired into AuthContext (see the table in 6.8).

```
// Accounts and the current session.
import {
  createUserWithEmailAndPassword,
  signInWithEmailAndPassword,
  signOut,
} from "firebase/auth";
import { doc, DocumentSnapshot, getDoc, writeBatch } from "firebase/firestore";

import { ApiError } from "@/api/ApiError";
import { auth, db, toApiError } from "@/api/firebase";
import { LoginResponse } from "@/types/api";
import { User, UserSettings } from "@/types/models";

// New accounts start in Swedish with notifications off; turning one on asks for permission.
const defaultSettings: UserSettings = {
  language: "sv",
  theme: "system",
  notifications: { itemAdded: false, itemCompleted: false },
};

export const toUser = (snap: DocumentSnapshot): User => {
  const data = snap.data()!;
  return { id: snap.id, name: data.name, email: data.email, settings: data.settings };
};

async function loadUser(uid: string): Promise<User> {
  const snap = await getDoc(doc(db, "users", uid));
  if (!snap.exists()) throw new ApiError("errors.invalidSession");
  return toUser(snap);
}

export async function login(email: string, password: string): Promise<LoginResponse> {
  try {
    const { user } = await signInWithEmailAndPassword(auth, email.trim(), password);
    return { token: await user.getIdToken(), user: await loadUser(user.uid) };
  } catch (error) {
    throw toApiError(error);
  }
}

// Firebase stores and refreshes the session itself. The token AuthContext saved only
// tells it that someone was logged in; wait for Firebase to restore that session.
export async function getCurrentUser(_token: string): Promise<User> {
  await auth.authStateReady();
  if (!auth.currentUser) throw new ApiError("errors.invalidSession");
  return loadUser(auth.currentUser.uid);
}

// New: creates the account plus its private user document and public profile.
export async function register(name: string, email: string, password: string): Promise<LoginResponse> {
  try {
    const { user } = await createUserWithEmailAndPassword(auth, email.trim(), password);
    const profile: User = { id: user.uid, name, email: email.trim(), settings: defaultSettings };
    const batch = writeBatch(db);
    batch.set(doc(db, "users", user.uid), { name, email: profile.email, settings: defaultSettings });
    batch.set(doc(db, "profiles", user.uid), { name });
    await batch.commit();
    return { token: await user.getIdToken(), user: profile };
  } catch (error) {
    throw toApiError(error);
  }
}
```

```
// New: ends the Firebase session. AuthContext.logout must call it.
export async function logout(): Promise<void> {
  await signOut(auth);
}
```

6.5 api/settings.ts and api/users.ts

```
// api/settings.ts
import { doc, getDoc, setDoc } from "firebase/firestore";

import { toUser } from "@/api/auth";
import { db } from "@/api/firebase";
import { User, UserSettings } from "@/types/models";

// Saves the given settings and returns the updated user.
export async function updateSettings(userId: string, settings: Partial<UserSettings>): Promise<User> {
  const ref = doc(db, "users", userId);
  // merge: only the settings passed in change, like the mock's spread.
  await setDoc(ref, { settings }, { merge: true });
  return toUser(await getDoc(ref));
}
```

`getUsers()` is the one function that can't keep its exact meaning. In the mock it returns every user, including emails and settings. With households, a user may only see the names of people they share a list with, and only names. Suggested change, which needs agreement first since it alters a signature: return a new `Member` type (`{ id: string; name: string }`, added to types/models.ts). Its only caller, the notification hook, only reads `name`, and is rewritten in chapter 7 anyway.

```
// api/users.ts
import { collection, doc, getDoc, getDocs, query, where } from "firebase/firestore";

import { currentUid, db } from "@/api/firebase";
import { Member } from "@/types/models";

// Everyone who shares at least one list with me. Used to put names on notifications.
export async function getUsers(): Promise<Member[]> {
  const lists = await getDocs(
    query(collection(db, "lists"), where("members", "array-contains", currentUid())),
  );
  const ids = [...new Set(lists.docs.flatMap((list) => list.data().members as string[]))];
  const profiles = await Promise.all(ids.map((id) => getDoc(doc(db, "profiles", id))));
  return profiles
    .filter((profile) => profile.exists())
    .map((profile) => ({ id: profile.id, name: profile.data()!.name }));
}
```

6.6 api/lists.ts

```
// The shopping lists the logged-in user is a member of.
import {
  addDoc, arrayUnion, collection, doc, getDoc, getDocs,
  orderBy, query, serverTimestamp, updateDoc, where, writeBatch,
} from "firebase/firestore";

import { ApiError } from "@/api/ApiError";
import { currentUid, db, toApiError } from "@/api/firebase";
import { ShoppingList } from "@/types/models";

const myListsQuery = (uid: string) =>
```

```
  query(collection(db, "lists"), where("members", "array-contains", uid), orderBy("createdAt"));

export async function getLists(): Promise<ShoppingList[]> {
  const snap = await getDocs(myListsQuery(currentUid()));
  return snap.docs.map((d) => ({ id: d.id, name: d.data().name }));
}

export async function createList(name: string): Promise<ShoppingList> {
  const uid = currentUid();
  const ref = await addDoc(collection(db, "lists"), {
    name,
    ownerId: uid,
    members: [uid],
    createdAt: serverTimestamp(),
  });
  return { id: ref.id, name };
}

// Deletes the list together with all its items. Firestore has no cascading
// deletes, so the items go in the same batch (max 500 writes: plenty for a list).
export async function deleteList(listId: string): Promise<void> {
  const list = await getDoc(doc(db, "lists", listId));
  if (!list.exists()) throw new ApiError("errors.listNotFound");
  const items = await getDocs(query(collection(db, "items"), where("listId", "==", listId)));
  const batch = writeBatch(db);
  items.docs.forEach((item) => batch.delete(item.ref));
  batch.delete(list.ref);
  await batch.commit();
}

// New: creates a code others can use to join the list. The document id is the code.
export async function createInvite(listId: string, listName: string): Promise<string> {
    try {
        const ref = await addDoc(collection(db, "invites"), {
            listId,
            listName,
            createdBy: currentUid(),
            createdAt: serverTimestamp(),
        });
        return ref.id;
    } catch (error) {
        throw toApiError(error);
    }
}
// New: joins the list an invite code points to.
export async function acceptInvite(code: string): Promise<ShoppingList> {
    try {
        const invite = await getDoc(doc(db, "invites", code.trim()));
        if (!invite.exists()) throw new ApiError("errors.inviteNotFound");
        const { listId, listName } = invite.data();
        const list: ShoppingList = { id: listId, name: listName };
        // Readable only by members: if this works, I'm already in the list. Joining again would
        // be rejected by the rules, since it wouldn't add anyone.
        const existing = await getDoc(doc(db, "lists", listId)).catch(() => null);
        if (existing?.exists()) return list;
        // The rules check that this adds exactly me, with a valid invite for this list.
        await updateDoc(doc(db, "lists", listId), { members: arrayUnion(currentUid()), invite: invite.id });
        return list;
    } catch (error) {
        throw toApiError(error);
    }
}
```

Sharing needs some UI that doesn't exist yet: a "Share" action per list on the lists screen that calls `createInvite` and opens the system share sheet (React Native's built-in `Share`, no new dependency) with the code or a link like `https://<you>.github.io/vadfanskavihandla/join?code=…`, and a join screen (app/join.tsx) with a code field that calls `acceptInvite`. Every new text goes into both en.json and sv.json.

6.7 api/items.ts

Items live in a top-level `items` collection with a `listId` field, rather than in a sub-collection under each list. That keeps every current signature: `toggleItem(itemId, userId)` can find an item from its id alone.

```
// Items on a shopping list.
import {
  addDoc, collection, doc, DocumentSnapshot, getDoc, getDocs,
  orderBy, query, serverTimestamp, updateDoc, where, writeBatch,
} from "firebase/firestore";

import { ApiError } from "@/api/ApiError";
import { db } from "@/api/firebase";
import { AddItemRequest } from "@/types/api";
import { Item } from "@/types/models";

export const toItem = (snap: DocumentSnapshot): Item => {
  const data = snap.data()!;
  return {
    id: snap.id,
    listId: data.listId,
    name: data.name,
    done: data.done,
    addedBy: data.addedBy,
    amount: data.amount,
    category: data.category,
  };
};

// Oldest first: sortItems keeps that order within each category.
export const itemsQuery = (listId: string) =>
  query(collection(db, "items"), where("listId", "==", listId), orderBy("createdAt"));

export async function getItems(listId: string): Promise<Item[]> {
  const snap = await getDocs(itemsQuery(listId));
  return snap.docs.map(toItem);
}

export async function addItem(request: AddItemRequest, userId: string): Promise<Item> {
  const ref = await addDoc(collection(db, "items"), {
    ...request,
    done: false,
    addedBy: userId,
    updatedBy: userId,
    createdAt: serverTimestamp(),
  });
  return { ...request, id: ref.id, done: false, addedBy: userId };
}

export async function toggleItem(itemId: string, userId: string): Promise<Item> {
  const ref = doc(db, "items", itemId);
  const snap = await getDoc(ref);
  if (!snap.exists()) throw new ApiError("errors.itemNotFound");
  const item = toItem(snap);
  await updateDoc(ref, { done: !item.done, updatedBy: userId });
  return { ...item, done: !item.done };
}

export async function setAllDone(listId: string, done: boolean, userId: string): Promise<Item[]> {
  const snap = await getDocs(itemsQuery(listId));
  const batch = writeBatch(db);
  snap.docs
    .filter((d) => d.data().done !== done)
    .forEach((d) => batch.update(d.ref, { done, updatedBy: userId }));
  await batch.commit();
  return snap.docs.map((d) => ({ ...toItem(d), done }));
}
```

```
export async function removeDone(listId: string): Promise<void> {
  const snap = await getDocs(
    query(collection(db, "items"), where("listId", "==", listId), where("done", "==", true)),
  );
  const batch = writeBatch(db);
  snap.docs.forEach((d) => batch.delete(d.ref));
  await batch.commit();
}
```

6.8 Changes outside api/

The point of the api/ seam is that this list stays short:

File	Change
context/AuthContext.tsx	`logout` calls `authApi.logout()` before clearing the token. Keep helpers/storage.ts for now: the saved token still tells AuthContext whether to try restoring a session. Later, `getToken` can be replaced by `onAuthStateChanged` exposed from api/auth.ts.
app/login.tsx	Add a "Create account" mode that calls `register`. Remove the test-account hint.
app/lists.tsx, app/join.tsx	Share and join, as described in 6.6.
types/models.ts	Add `Member` and `ItemChange` (chapter 7).
i18n/locales/*.json	New error keys and all new UI text, in both languages.
hooks/useItems.ts	Nothing. Same functions, same signatures.
context/ListsContext.tsx	Nothing in this chapter; chapter 7 adds the live subscription.

Work through the files in this order, testing after each: firebase.ts, auth.ts (log in with a real account), settings.ts, lists.ts, items.ts, users.ts. Until a file is rewritten it keeps re-exporting the mock, so the app runs throughout. Mixing is awkward for lists and items (they reference each other's ids), so switch those two together.

7. Live updates

Firestore can turn any query into a live listener with `onSnapshot`. Two small additions to api/ expose that without leaking Firestore types to the rest of the app. A hook then writes the results straight into the TanStack Query cache, so every screen that already uses `useItems` or `useLists` updates by itself.

7.1 Subscribe functions in api/

```
// In api/lists.ts:
// Calls onChange with my lists whenever one is created, renamed, deleted, or I'm added to one.
// Returns a function that stops listening.
export function subscribeToLists(userId: string, onChange: (lists: ShoppingList[]) => void): () => void {
    return onSnapshot(
        myListsQuery(userId),
        (snap) => onChange(snap.docs.map((d) => ({ id: d.id, name: d.data().name }))),
        // Denied for a moment while logging out; the listener is stopped right after.
        () => {},
    );
}
```

```
// In api/items.ts:
// Calls onChange with the list's items whenever anyone changes them, plus what other
// people just did (for notifications). Returns a function that stops listening.
export function subscribeToItems(
    listId: string,
    userId: string,
    onChange: (items: Item[], changes: ItemChange[]) => void,
): () => void {
    let isFirstSnapshot = true;
    return onSnapshot(itemsQuery(listId), (snap) => {
        // The first snapshot is everything already on the list, not news.
        const changes = isFirstSnapshot
            ? []
            : snap.docChanges().flatMap((change): ItemChange[] => {
                const data = change.doc.data();
                if (data.updatedBy === userId) return [];
                const item = toItem(change.doc);
                if (change.type === "added") return [{ type: "itemAdded", item, userId: data.updatedBy }];
                if (change.type === "modified" && data.done)
                    return [{ type: "itemCompleted", item, userId: data.updatedBy }];
                return [];
            });
        isFirstSnapshot = false;
        onChange(snap.docs.map(toItem), changes);
    },
    // Denied once the list is deleted or I log out. The lists listener drops the list
    // and this listener is stopped, so there is nothing to do.
    () => {});
}
```

The empty error callbacks matter: when a list is deleted, or while logging out, Firestore denies the listener for a moment before the app stops it. Without a callback, that shows up as an uncaught error.

`ItemChange` goes in types/models.ts: `{ type: ActivityType; item: Item; userId: string }`. It replaces `ActivityEvent`, whose increasing numeric id only existed for polling.

7.2 A hook that feeds the cache: hooks/useLiveUpdates.ts

This replaces hooks/useActivityNotifications.ts and is mounted in the same place (`RootStack` in app/_layout.tsx).

```
import { useQuery, useQueryClient } from "@tanstack/react-query";
import { useEffect, useEffectEvent } from "react";
import { useTranslation } from "react-i18next";

import { subscribeToItems } from "@/api/items";
import { subscribeToLists } from "@/api/lists";
import { getUsers } from "@/api/users";
import { useAuth } from "@/context/AuthContext";
import { useLists } from "@/context/ListsContext";
import { canReceivePush, showNotification } from "@/helpers/notifications";
import { ItemChange } from "@/types/models";

// Keeps lists and items live for every list the user is in, and turns other people's
// changes into notifications if the user has turned them on. Mount once.
// Installed builds get those as push notifications from the server instead, so they aren't shown twice.
export function useLiveUpdates() {
  const { t } = useTranslation();
  const { user } = useAuth();
  const { lists } = useLists();
  const queryClient = useQueryClient();
  const userId = user?.id;

  const { data: members } = useQuery({ queryKey: ["users"], queryFn: getUsers, enabled: !!userId });

  // Reads the latest settings, names and translations without re-subscribing.
  const notify = useEffectEvent((change: ItemChange) => {
    if (canReceivePush || !user?.settings.notifications[change.type]) return;
    // Someone who just joined isn't in the cached members yet.
    if (!members?.some((m) => m.id === change.userId)) queryClient.invalidateQueries({ queryKey: ["users"] });
    const who = members?.find((m) => m.id === change.userId)?.name ?? "";
    const listName = lists.find((l) => l.id === change.item.listId)?.name ?? "";
    showNotification(listName, t(`notifications.${change.type}`, { user: who, item: change.item.name }));
  });

  useEffect(() => {
    if (!userId) return;
    return subscribeToLists(userId, (fresh) => queryClient.setQueryData(["lists"], fresh));
  }, [userId, queryClient]);

  // One listener per list. A string key, so a new array with the same lists doesn't re-subscribe.
  const listIds = lists.map((l) => l.id).join(",");
  useEffect(() => {
    if (!userId || !listIds) return;
    const stops = listIds.split(",").map((listId) =>
      subscribeToItems(listId, userId, (items, changes) => {
        queryClient.setQueryData(["items", listId], items);
        changes.forEach((change) => notify(change));
      }),
    );
    return () => stops.forEach((stop) => stop());
  }, [userId, listIds, queryClient]);
}
```

With this in place:

- The `onSuccess: refresh` calls in useItems and ListsContext still work but are no longer needed: Firestore applies your own writes to the listener instantly (before the server even confirms). Removing them saves a re-read of the whole list per change. Keep the optimistic toggle as it is.
- The polling, `getActivity`, api/activity.ts and the `startMockActivity` effect all go.
- This hook's notifications only show while the app is open. That's enough on web and in Expo Go. Installed builds get

real push from chapter 8 instead, which is why `notify` returns early when `canReceivePush` is true: otherwise each change would notify twice.

7.3 Offline

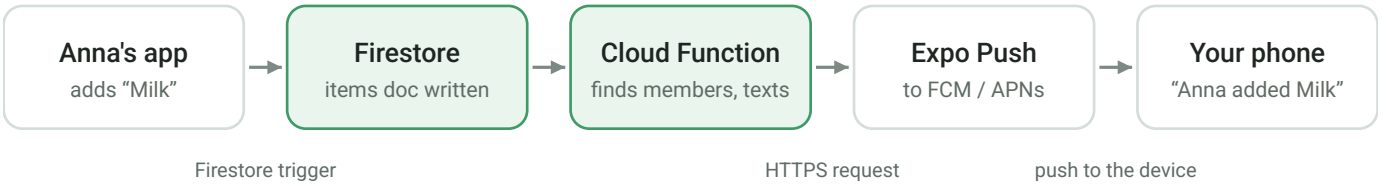
Firestore queues writes made without signal and sends them later, and listeners show the local change immediately. On phones the cache is in memory, so it survives losing signal but not closing the app. On web you can opt into a persistent IndexedDB cache with `persistentLocalCache()` in `initializeFirestore`.

7.4 Remove the mock

When every api/ file uses Firebase and the app works on two devices logged in as different users: delete mock/, api/activity.ts, hooks/useActivityNotifications.ts and the test-account hint, and update the README's "Get started" and "Connecting a real backend" sections.

8. Push notifications

Live updates only reach an open app. Push notifications arrive even when it's closed, so they are sent from the server.



8.1 How it fits together

- Push token. After login, the app asks the phone for an Expo push token (an address for this app on this phone) and saves it in `users/{uid}/pushTokens/{token}`.
- Trigger. A Cloud Function runs whenever a document in `items` is created or changed.
- Function. It works out what happened (added, or checked off) and who did it (`updatedBy`), finds the list's other members, checks each one's notification settings and language, and sends the message to their tokens.
- Expo Push Service. Expo's free service takes the message and a token and forwards it to Google (FCM) or Apple (APNs). On Android, FCM credentials from this same Firebase project are uploaded to EAS once (chapter 10.4).

8.2 In the app

1. Add the expo-notifications config plugin to app.json and run `eas init` so the app has an EAS `projectId` (chapter 10.2).
2. In helpers/notifications.ts: `canReceivePush` is true only in an installed build (not web, not Expo Go) that has a `projectId`. `getPushToken()` goes through the existing `loadNotifications()`, so Expo Go stays safe, and returns `null` unless push is available and allowed.
3. api/push.ts: `savePushToken(userId, token)` and `removePushToken(userId, token)` write and delete `users/{uid}/pushTokens/{token}` with `{ platform, updatedAt }`.
4. hooks/usePushRegistration.ts, mounted next to useLiveUpdates: registers the token after login and again when a notification setting is switched on.
5. AuthContext's `logout` removes the token before `signOut`, while the rules still know who you are, so a phone that changes hands stops getting the previous user's notifications.

8.3 The Cloud Function

In functions/src/index.ts. The notification texts come from the app's own locale files, so there is still only one place to translate. The build script in functions/package.json copies them in first, and the function reads them at runtime. Add `locales/` to functions/.gitignore, since it's generated:

```
"copy-locales": "mkdir -p locales && cp ../i18n/locales/*.json locales/",
"build": "npm run copy-locales && tsc",
```

The function code follows the Google style that `firebase init` sets up for functions/ (2-space indent, 80-column lines, JSDoc on functions), because `firebase deploy` runs that lint first.

```
/**
 * Sends push notifications when someone adds or checks off an item, to the
 * list's other members who have that notification switched on.
 */
import {initializeApp} from "firebase-admin/app";
import {getFirestore} from "firebase-admin/firestore";
import {setGlobalOptions} from "firebase-functions";
import * as logger from "firebase-functions/logger";
import {onDocumentWritten} from "firebase-functions/v2/firestore";
import {readFileSync} from "fs";
import {join} from "path";

initializeApp();
// Cloud Functions isn't offered in europe-north2 (Stockholm), where the
// database is. Finland is the nearest region that has it; the Firestore
// trigger itself still lives with the database. Few instances: this only
// reacts to a household's taps.
setGlobalOptions({region: "europe-north1", maxInstances: 5});
const db = getFirestore();

type ChangeType = "itemAdded" | "itemCompleted";
type Language = "en" | "sv";

interface Texts {
  notifications: Record<ChangeType, string>;
}

/**
 * Reads the app's own translations, copied into locales/ by the build
 * script, so notification texts are only translated in one place.
 * @param {Language} language The language code.
 * @return {Texts} The parsed locale file.
 */
function loadTexts(language: Language): Texts {
  const file = join(__dirname, "..", "locales", `${language}.json`);
  return JSON.parse(readFileSync(file, "utf8"));
}
const texts: Record<Language, Texts> = {
  en: loadTexts("en"),
  sv: loadTexts("sv"),
};

interface PushMessage {
  to: string;
  title: string;
  body: string;
  sound: "default";
  channelId: "default";
}

export const notifyOnItemChange = onDocumentWritten(
  "items/{itemId}",
  async (event) => {
    const before = event.data?.before.data();
    const after = event.data?.after.data();
    if (!after) return; // Deleted.

    let type: ChangeType | null = null;
    if (!before) type = "itemAdded";
    else if (!before.done && after.done) type = "itemCompleted";
    if (!type) return;

    const actorId: string = after.updatedBy;
    const list = await db.doc(`lists/${after.listId}`).get();
    if (!list.exists) return;
    const actor = await db.doc(`profiles/${actorId}`).get();
    const members: string[] = list.get("members") ?? [];
    const recipients = members.filter((id) => id !== actorId);
```

```
    // Each message remembers whose token it is, so dead tokens can be removed.
    const sends: { userId: string; message: PushMessage }[] = [];
    for (const userId of recipients) {
      const user = await db.doc(`users/${userId}`).get();
      if (!user.get(`settings.notifications.${type}`)) continue;
      const language: Language =
        user.get("settings.language") === "en" ? "en" : "sv";
      const body = texts[language].notifications[type]
        .replace("{{user}}", actor.get("name") ?? "")
        .replace("{{item}}", after.name);
      const tokens = await db.collection(`users/${userId}/pushTokens`).get();
      tokens.forEach((token) => sends.push({
        userId,
        message: {
          to: token.id,
          title: list.get("name"),
          body,
          sound: "default",
          channelId: "default",
        },
      }));
    }
    if (sends.length === 0) return;

    const response = await fetch("https://exp.host/--/api/v2/push/send", {
      method: "POST",
      headers: {"Content-Type": "application/json"},
      body: JSON.stringify(sends.map((send) => send.message)),
    });
    if (!response.ok) {
      logger.error("Expo push failed", response.status, await response.text());
      return;
    }

    // Tickets come back in the same order as the messages. Forget tokens
    // for phones where the app was uninstalled.
    const {data: tickets} = await response.json() as {
      data: { status: string; details?: { error?: string } }[];
    };
    await Promise.all(
      sends
        .filter((_send, i) =>
          tickets[i]?.details?.error === "DeviceNotRegistered")
        .map(({userId, message}) =>
          db.doc(`users/${userId}/pushTokens/${message.to}`).delete()),
    );
  },
);
```

"Check all" on a long list sends one notification per item; batching those per list and person is a good later improvement.

Deploy with `firebase deploy --only functions`. This is the step that needs the Blaze plan (chapter 12).

8.4 Region

A 2nd-gen Firestore trigger has two parts: the trigger, which always lives in the database's location, and the function, which runs wherever `region` says. They don't have to match. This project's database is in europe-north2 (Stockholm), where Cloud Functions isn't offered: deploying there fails with "Location europe-north2 is not found or access is unauthorized". So the function runs in europe-north1 (Finland), the nearest region that has it, and Firebase creates the trigger in europe-north2 by itself. The extra hop costs milliseconds.

If your database is in a region that has Cloud Functions, use that same region and skip the hop. Check the current list at firebase.google.com/docs/functions/locations.

8.5 First deploy: what can go wrong

The first function deploy on a project switches on several Google Cloud services (Cloud Build, Artifact Registry, Cloud Run, Eventarc, Pub/Sub) and creates their service accounts. Expect one or two failed attempts:

Error says	What to do
"must be on the Blaze (pay-as-you-go) plan"	Set a budget alert, then upgrade (chapter 12).
"We failed to modify the IAM policy for the project"	The new service accounts aren't ready yet. Wait about 5 minutes and deploy again. If it keeps failing, check that your account is an Owner of the project, then grant the three roles the CLI prints, in Google Cloud Console › IAM (tick "Include Google-provided role grants").
"Location … is not found or access is unauthorized"	Cloud Functions isn't offered in that region. Pick another `region` (see 8.4).
"Permission denied while using the Eventarc Service Agent" / "first time using 2nd gen functions"	Permissions are still propagating. Wait 5–15 minutes and deploy again. If it persists, give `service-<project-number>@gcp-sa-eventarc.iam.gserviceaccount.com` the Eventarc Service Agent role.
ESLint: "Invalid option '--ext'"	ESLint found the app's eslint.config.js in the folder above and switched to the new config format. Make the functions lint script `ESLINT_USE_FLAT_CONFIG=false eslint --ext .js,.ts .`

> **Push needs a real build, not Expo Go**
>
> Since Expo SDK 53, remote push notifications don't work in Expo Go on Android, so test them in a development or preview build (chapter 10). On iPhone, push also requires the paid Apple Developer Program. Web gets the in-app notifications from chapter 7; real web push (FCM web push with a service worker) is possible later.

9. Testing with the emulators

The Local Emulator Suite runs Auth, Firestore and Functions on your machine. Use it for every rules change and for developing the Cloud Function. It's free and can't damage real data.

```
firebase emulators:start --import=./emulator-data --export-on-exit
# UI with data, users and function logs: http://localhost:4000
```

- App against the emulators: set `EXPO_PUBLIC_FIREBASE_EMULATOR_HOST` in .env.local to `localhost` for web, or `10.0.2.2` for the Android emulator, and restart Expo.
- Two users on one machine: log in as one user in the browser and another in the Android emulator. That's also how CLAUDE.md's "works on web and in the Android emulator" check covers sharing.
- Rules unit tests (optional): `@firebase/rules-unit-testing` in the functions/ project lets you write tests like "a non-member can't read items". Worth it once the rules grow.

10. Getting the app onto phones

Until now the app has run in Expo Go or a browser. To use it day to day, build it into a real app with its own icon. EAS Build does this in the cloud, so you don't need Android Studio or a Mac to build.

10.1 The kinds of builds

Build	What it is	Use it for
Expo Go	Expo's own app that loads your code. No install, but no remote push and no own icon.	Quick development. Firebase works here.
Development build	Your own app with developer tools inside. Loads code live from your computer.	Developing push and other native features.
Preview build	Your finished app as an installable file, shared directly.	Using it yourselves and testing with others.
Store build	Uploaded to Google Play / App Store.	When other households should install it from a store.

10.2 One-time setup

```
npm install -g eas-cli
eas login              # free Expo account
eas init               # links the project, adds the projectId to app.json
eas build:configure    # creates eas.json
```

In eas.json, make the preview profile produce an installable APK:

```
"preview": {
  "distribution": "internal",
  "android": { "buildType": "apk" }
}
```

EAS builds don't see .env.local. Add the `EXPO_PUBLIC_FIREBASE_*` values as EAS environment variables (`eas env:create`, visibility "plain text" is fine since they're public) for the preview and production environments.

10.3 Android

1. Set `android.package` in app.json, for example `"se.<you>.vadfanskavihandla"`. It can't be changed after release.
2. Run `eas build --profile preview --platform android`. It builds in the cloud and gives you a link and QR code.
3. Open the link on each Android phone and install. The phone asks you to allow installs from that source.

10.4 Android push credentials

Expo's push service delivers to Android through FCM, which is part of your Firebase project:

1. In the Firebase console, add an Android app with the same package name. Download google-services.json and set `android.googleServicesFile` in app.json. (It isn't secret, but you can keep it out of git and provide it as an EAS file variable.)

2. In Project settings › Service accounts, generate a private key. Upload it with `eas credentials` › Android › Google Service Account › FCM V1. Then delete the downloaded file.

> **The service account key is a real secret**
>
> It gives full admin access to the Firebase project, bypassing every rule. It goes to EAS only. Never in the app, the repo, or a chat. The .gitignore doesn't cover .json files, so be careful where you save it.

10.5 iPhone (later)

Apple requires the Apple Developer Program ($99/year) to install your own builds. Then either register each iPhone with `eas device:create` and make an internal build, or use TestFlight (`eas submit`). For iOS push, EAS sets up the APNs key for you during the build. The iPhone can also use the web version for free in the meantime (chapter 11).

10.6 Updating the app later

Most changes are JavaScript only (screens, logic, styles, api/). Those reach installed apps with EAS Update (`eas update`) without reinstalling. A new build is only needed when you add a package with native code or change app.json.

11. Web on GitHub Pages

The same code exported as a static website. Firebase works the same way in the browser, so nothing in api/ changes.

11.1 Build settings

The site will live at `https://<you>.github.io/vadfanskavihandla/`, not at the domain root, so tell Expo Router the base path in app.json:

```
"experiments": {
  "typedRoutes": true,
  "reactCompiler": true,
  "baseUrl": "/vadfanskavihandla"
}
```

Then build:

```
npx expo export --platform web     # writes dist/
touch dist/.nojekyll               # otherwise Pages drops the _expo/ folder
```

GitHub Pages runs Jekyll by default, and Jekyll ignores folders starting with an underscore, which is where Expo puts the JavaScript. The empty `.nojekyll` file turns that off.

11.2 Deploy with GitHub Actions

Add a workflow (.github/workflows/web.yml) that builds and publishes on every push to main. The Firebase values go in the repository's Settings › Secrets and variables › Actions › Variables (they're public, so variables rather than secrets is fine):

```
name: Web
on:
  push:
    branches: [main]
permissions:
  contents: read
  pages: write
  id-token: write
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: github-pages
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22, cache: npm }
      - run: npm ci
      - run: npx expo export --platform web && touch dist/.nojekyll
        env:
          EXPO_PUBLIC_FIREBASE_API_KEY: ${{ vars.FIREBASE_API_KEY }}
          EXPO_PUBLIC_FIREBASE_AUTH_DOMAIN: ${{ vars.FIREBASE_AUTH_DOMAIN }}
          EXPO_PUBLIC_FIREBASE_PROJECT_ID: ${{ vars.FIREBASE_PROJECT_ID }}
          EXPO_PUBLIC_FIREBASE_APP_ID: ${{ vars.FIREBASE_APP_ID }}
      - uses: actions/upload-pages-artifact@v3
        with: { path: dist }
      - uses: actions/deploy-pages@v4
```

In the repo's Settings › Pages, set the source to "GitHub Actions". In the Firebase console under Authentication › Settings ›

Authorized domains, add `<you>.github.io`; email/password works without it, but Google sign-in later won't.

> **Private repositories**
>
> GitHub Pages is free for public repositories. Publishing from a private one needs a paid GitHub plan. The code being public is fine: nothing in it is secret, and the data is protected by the rules.

12. Hosting and what it costs

Everything here is a managed service; there is no server for you to run, update or back up by hand.

Service	What runs there	Price (Oct 2026)
Firebase Spark	Auth, Firestore, rules, realtime. Everything except Cloud Functions.	Free: 50,000 reads and 20,000 writes per day, 1 GiB stored, 50,000 monthly active users for email sign-in. No card needed.
Firebase Blaze	Adds Cloud Functions (push). Same free quotas as Spark, then pay per use.	Needs a card. Functions include a free monthly allowance far above this app's needs. Expect 0 kr, or a few kronor for function image storage.
EAS (Expo)	Builds the Android and iOS apps; app updates.	Free plan with a monthly build allowance (15 Android + 15 iOS when checked).
GitHub Pages	The web version.	Free for public repositories.
Apple Developer Program	Needed to install on iPhones and for iOS push.	$99 / year
Google Play	Only if you publish to the store; preview APKs don't need it.	$25 once

To get a feel for the free tier: one list of 30 items opened by four people is 120 reads; each added item is one write plus one read per open listener. A busy household stays well under 1 % of the daily quota.

> **Set a budget alert before switching to Blaze**
>
> In Google Cloud Billing, create a budget (for example 50 kr/month) with email alerts. A bug that writes in a loop is the realistic way to get a bill, and the alert tells you within hours. Also run `firebase functions:artifacts:setpolicy` once, so old function images are cleaned up instead of slowly accumulating storage charges.

Setup	You pay for	Per year
Android + web, no push	Nothing: Spark plan, free builds, APK installed directly, GitHub Pages.	0 kr
Android + web, with push	Blaze plan, within free quota.	≈ 0 kr
Including iPhone (native app)	Apple Developer Program.	$99
Published to Google Play	One-time Play registration.	$25 once

13. Step-by-step roadmap

Each phase leaves something that works, so you can stop at any point and still use the app.

13.1 Phase 0: Agree on the changes

- Update CLAUDE.md from Supabase to Firebase (chapter 4.3).
- Approve the new dependencies: `firebase` and `@react-native-async-storage/async-storage`.
- Agree on the api/ additions (`register`, `logout`, `createInvite`, `acceptInvite`, `subscribeToLists`, `subscribeToItems`, push token functions) and the one signature change (`getUsers` returning `Member[]`).

13.2 Phase 1: Backend (one evening)

- Create the Firebase project, enable Email/Password, create Firestore in production mode, and note its location (5.1).
- `firebase init`, write firestore.rules and firestore.indexes.json, deploy them (5.2–5.4).
- Try the rules in the emulator with two test users (chapter 9).

13.3 Phase 2: Connect the app

- Install the SDK, add .env.local and api/firebase.ts (6.1–6.3).
- Rewrite api/auth.ts, add sign-up to the login screen, wire `logout` into AuthContext.
- Rewrite settings, lists + items together, then users. Test after each.

13.4 Phase 3: Sharing

- Share action and join screen with `createInvite` / `acceptInvite`, texts in both languages.
- Test: two accounts on web + emulator, share a list, add items from both sides.

13.5 Phase 4: Live updates and cleanup

- Add the subscribe functions and useLiveUpdates; remove polling and the activity API (chapter 7).
- Delete mock/ and update the README.

13.6 Phase 5: Web on GitHub Pages

- baseUrl, .nojekyll, Actions workflow, Pages source, authorized domain (chapter 11).

13.7 Phase 6: Android build

- EAS setup, environment variables, preview build, install on your phones (chapter 10).

13.8 Phase 7: Push notifications

- Budget alert, then switch to Blaze. Android app + FCM credentials in EAS (10.4).
- Push token registration in the app, the Cloud Function in a region that offers Cloud Functions, deploy (chapter 8; expect a retry or two on the first deploy, 8.5). New build, since expo-notifications needs native code.

13.9 Phase 8: Nice to have

- Leave a list, rename a list, remove members (the rules already allow leaving and renaming).
- Delete account. Required by Google Play if you publish there; also just good manners.
- App Check, so only your real app can call the backend.
- Scheduled Firestore backups (Blaze) or a periodic `gcloud firestore export`.
- Persistent offline cache on web (7.3).

14. Security checklist

Check	Done
Firestore was created in production mode; test-mode rules were never deployed.	
firestore.rules is in the repo, deployed, and nothing in it says `allow read, write: if true`.	
The rules were tested in the emulator: a non-member can't read or write a list's items, and nobody can forge `addedBy` / `updatedBy`.	
No Firebase calls outside api/. The Cloud Function is the only code that bypasses rules, and it only reads what it needs to send a notification.	
The service account key exists only in EAS. Not in the app, the repo, or on disk.	
.env.local is not committed (.gitignore already covers it).	
Email enumeration protection is on.	
A Google Cloud budget alert is set before switching to Blaze.	
Invite codes are random document ids (never sequential or guessable), and the invites collection can't be listed.	
You have a recent Firestore export somewhere safe.	

15. Glossary

Term	Meaning
api/ seam	The folder that is the app's only contact with the backend. Swapping backends means rewriting it, nothing else.
Blaze / Spark	Firebase's pay-as-you-go and free plans. Blaze keeps Spark's free quotas.
Cloud Function	Server code that Firebase runs when something happens, like an item being written.
Collection / document	Firestore's units: a collection (lists) holds documents (one list), each a set of fields.
Composite index	An index over several fields, needed for queries that filter on one field and sort on another.
EAS	Expo Application Services: builds, submits and updates the app.
Expo Go	Expo's app for running your code during development without building it.
FCM / APNs	Google's and Apple's push notification services.
onSnapshot	Firestore's realtime listener: calls you with fresh results whenever the data changes.
Push token	An address for the app on one specific phone, used to send it notifications.
Security Rules	Firestore's access rules, deployed from firestore.rules. The equivalent of Row Level Security.
Service account key	An admin credential for the Firebase project. Bypasses all rules; never in the app.
TanStack Query cache	Where the app keeps server data. Live updates write into it, and screens re-render.

Sources

Prices and limits were checked in October 2026 and change over time. Check the providers' own pages before you rely on them.

- Firebase pricing and Spark/Blaze quotas: firebase.google.com/pricing.
- Firestore Security Rules and queries: firebase.google.com/docs/firestore/security/rules-query.
- Using Firebase with Expo: docs.expo.dev/guides/using-firebase; Expo's auth persistence notes: github.com/expo/fyi (firebase-js-auth-setup.md).
- getReactNativePersistence type gap: github.com/firebase/firebase-js-sdk/issues/9316.
- Expo push notifications and FCM V1 credentials: docs.expo.dev/push-notifications/overview.
- Expo Router static export and baseUrl: docs.expo.dev/router/reference/static-rendering; docs.expo.dev/more/expo-cli (export).
- GitHub Pages with Actions: docs.github.com/pages.
- EAS plans: expo.dev/pricing. Apple Developer Program: developer.apple.com. Google Play Console: play.google.com/console.